

Hybrid A* and MPC-iLQR for Robust Differential-Drive Robot Navigation Under Noise and Model Mismatch

Reddhi Das

Department of Electrical and Computer Engineering
North Carolina State University, Raleigh, NC, USA
Email: rdas5@ncsu.edu

Abstract—This paper presents a complete planning and control system for differential-drive robot navigation in cluttered environments affected by noise and model inaccuracies. The approach begins with a Hybrid A* planner, which produces a feasible geometric path using an 8-connected grid, discrete heading bins, and an occupancy map expanded to account for the robot's physical size. This global plan is then followed by a receding-horizon controller based on Model Predictive Control (MPC) and iterative Linear Quadratic Regulation (iLQR). The controller tracks a dense nominal trajectory while also incorporating soft obstacle-avoidance terms, penalties on sudden control changes, and automatic replanning whenever the robot drifts too far from the expected path. To realistically evaluate performance, the robot model includes actuator lag, mismatches in linear and angular motion, and additive state noise. A standard PID controller is used as a baseline for comparison. Simulation results show that MPC-iLQR responds more intelligently near obstacles and remains stable despite significant mismatch, whereas PID produces smoother but less predictive behavior. Metrics such as tracking error, control effort, and the evolution of the MPC cost highlight the trade-offs between predictive optimal control and simple feedback laws. Overall, the work demonstrates a practical and flexible navigation framework that combines global planning, real-time optimal control, and disturbance-aware replanning for robust robot operation in uncertain settings

Index Terms—Hybrid A*, Model Predictive Control, iterative LQR, robot navigation, differential-drive robot, optimal control, replanning, obstacle avoidance, noise robustness, model mismatch

I. INTRODUCTION

Mobile robots must be safe in the real world despite obstacles, disruptions, and inevitable differences between the mathematical model and reality. This problem is further complicated in the case of differential-drive platforms due to drift in their dynamical models, actuator constraints, and nonholonomic constraints. When the robot encounters noise or an imbalance between the nominal and actual dynamics, classical methods such as grid-based searches, potential fields, and PID feedback can typically be problematic. Due to these limitations, the study has integrated local disturbance-compensated control strategies with credible global planning [1], [2].

Hybrid A* is widely popular as a global planner due to the fact that it has discrete heading states and can naturalistically be combined with occupancy-grid maps. It generates collision-

free paths even in congested spaces and fits well with the limited mobile robots [3], [4]. A geometric plan is, however, not sufficient to facilitate reliable navigation. A loss of actuator lag, unforeseen deviation, or external disturbances may make the robot track an alternative path to the nominal one, and the controller needs to manage these.

Model Predictive Control (MPC) has become an influential approach as a local motion control tool for the control of systems due to its explicit use of system dynamics, constraints, and multi-objective costs within a prediction horizon [5], [6]. Iterative Linear Quadratic Regulation (iLQR) is an approximation of nonlinear MPC that is computationally efficient and involves the linearization of the system about a nominal trajectory followed by a series of backwards and forward Riccati-type passes [7]. This renders iLQR appealing in real-time robotic applications that demand predictive behavior along with efficiency of computation.

Although these methods of predictive control offer these benefits, modeling errors and unmodeled actuator dynamics can nonetheless be sensitive. Strong and adaptive versions of MPC have been studied in previous studies to overcome these problems [8], [9]. Also, soft obstacle-avoidance expenses have been demonstrated to enhance safety by avoiding hazardous regions with not hard restrictions on the robot [10]. Their operation in harsh noise and mismatch conditions is, however, still an open question, particularly when it is combined with global planning and automatic replanning.

This work develops an entire navigation structure combining three key elements. The global Hybrid A* planner initially calculates a feasible geometric path by using a grid that is 8-connected, discrete heading bins, and obstacle inflation to take into consideration the geometry of the robot. Second, an MPC-iLQR controller is based on a receding horizon strategy, following a dense nominal path and involving soft obstacle penalties, control-smoothing terms, and autonomous replanning in cases when tracking error becomes too large. Third, by using a realistic simulation model, such as actuator lag, linear velocity scaling mismatch, heading bias, and additive noise, we can study the behavior of both predictive and reactive controllers in non-ideal settings.

In order to have a point of reference, we compare the MPC-iLQR controller with a basic PID controller, which is well-

known in practical use because of its simplicity. We have demonstrated that MPC-iLQR can predict potential obstacles better and manage mismatch better, whereas PID generates smoother, although less informed, motion. The important trade-offs between predictive optimal control and traditional reactive approaches are demonstrated using quantitative indicators like monitoring of RMSE, overall control effort, and the development of the MPC horizon cost.

Contributions. The key contributions of this work are

- a single planning and control platform integrating Hybrid A* global planning and MPC-iLQR local tracking;
- combining soft obstacle costs, control-rate penalties and deviation-induced replanning with an online receding-horizon controller;
- simulation of a realistic robot model of actuator lag, heading error, velocity error, and additive noise; and
- a comparative analysis indicating the effect of noise and mismatch on the performance of MPC-iLQR as well as PID.

II. SYSTEM MODEL AND PROBLEM FORMULATION

This work is aimed at controlling a differential-drive robot in a cluttered environment and considering noise, disturbance, and deviations between a perfect mathematical model and the real behavior of the robot. In order to develop a complete navigation framework, we initially define the model of the robot, the arrangement of the surroundings, and the exact problem we are going to resolve.

A. Robot Kinematics and Actuator Behavior

The robot is modeled as a differential-drive platform with state variables

$$(x, y, \theta),$$

representing its position and heading. The control inputs are the forward velocity v and angular velocity ω , which determine how the robot moves through the environment. Under the standard nonholonomic kinematic model [1], the nominal motion evolves as

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega.$$

In reality, the actual robot, however, does not act in the same manner as the nominal model. Motors have lag, wheel slips will result in a small heading error, and poor calibration can result in a difference between real velocity and commanded velocity. In order to measure the behavior of the controller to such conditions, we present the model of mismatch of the form:

$$v_{\text{true}} = \alpha_v v, \quad \omega_{\text{true}} = \omega + b_\theta,$$

and additive process noise,

$$x_{t+1} = f(x_t, u_t) + w_t, \quad w_t \sim \mathcal{N}(0, \Sigma),$$

allowing us to test how the MPC-iLQR and PID controllers behave when disturbances move the robot away from the predicted trajectory.

B. Environment Representation

The surrounding is modeled with a 2-dimensional occupancy grid, indicating free and occupied cells. This type of representation is common in mobile robot navigation because it balances simplicity with enough detail for effective planning [11]. Obstacles are inflated by a radius to take into consideration the footprint of the robot.

$$r_{\text{infl}} = r_{\text{robot}} + r_{\text{margin}}.$$

This inflation ensures that near-obstacle cells are treated as unsafe, preventing the robot from grazing walls or entering narrow passages. Occupancy-grid representations with inflation are widely used in practical planning systems [2].

These obstacles on the map are both circular and rectangular to reflect a difficult and real-world scenario. These are the barriers that cause the planner and controller to make significant choices regarding the method of maneuvering in the narrow corridors and evading dangerous maneuvers.

C. Navigation Objectives

The navigation task is defined as follows: given a start state x_{start} , goal state x_{goal} , and an occupancy grid map, the robot must reach the goal without colliding with obstacles while maintaining smooth motion and reasonable control effort.

The MPC-iLQR controller minimizes a finite-horizon cost:

$$J = \sum_{t=0}^{N-1} [(x_t - x_t^{\text{nom}})^\top Q (x_t - x_t^{\text{nom}}) + u_t^\top R u_t + c_{\text{obs}}(x_t)] + (x_N - x_N^{\text{nom}})^\top Q_f (x_N - x_N^{\text{nom}}),$$

where soft obstacle avoidance uses

$$c_{\text{obs}}(x) = \exp\left(-\frac{d(x)}{\sigma}\right).$$

where Q , R , and Q_f weight tracking accuracy and control effort. Soft obstacle avoidance is encoded using an exponential penalty [10]:

$$c_{\text{obs}}(x) = \exp\left(-\frac{d(x)}{\sigma}\right),$$

where $d(x)$ is the distance to the nearest obstacle.

The robot should also respond properly to deviations due to mismatch or noise and replan where required as opposed to compelling the controller to pursue an obsolete course.

D. Overall Problem Setup

We integrate three elements in order to achieve these objectives. First, hybrid A* gives the global geometrical path, which gives a sequence of reachable poses that do not violate the nonholonomic structure of the robot [3], [4]. The planner solves a graph search problem:

$$\min_{\pi} \sum_{(i,j) \in \pi} c(i,j),$$

where each node encodes a discretized (x, y, θ) state.

Second, the MPC-iLQR controller tracks a dense nominal trajectory derived from the planner. iLQR works by repeatedly linearizing the dynamics:

$$x_{t+1} \approx f(x_t, u_t) \approx A_t x_t + B_t u_t,$$

and performing backward Riccati passes to compute feedback and feedforward terms:

$$u_t = u_t^{\text{nom}} + k_t + K_t(x_t - x_t^{\text{nom}}),$$

as described in [5]–[7].

Lastly, in order to assess both predictive and reactive controllers under practical circumstances, the simulation includes actuator lag, velocity and heading mismatch, and additive noise.

III. GLOBAL PATH GENERATION USING HYBRID A*

In this section, the generation of the global path is discussed, and then the control is passed to the MPC-iLQR tracking stage. Although numerous grid-based planners have been developed, Hybrid A* is the most useful in nonholonomic robots, as it considers both the geometry of the world and the constraints of movement of the robot in the search. In our study, it offers a viable, collision-free route that observes the differential-drive kinematics.

A. Motivation for Hybrid A*

Traditional A* is effective in geometric grids, and the assumption is that the robot is free to move in whichever direction it chooses after selecting the grid cell. That is not the case with differential-drive robots. They are not able to turn or to move, neither sideways at all nor in place. Hybrid A* copes with these constraints by adding a discrete heading to every grid cell, and therefore the planner reasons over a state.

$$s = (x, y, \theta_h),$$

where θ_h is a quantized heading index (e.g., one of 16 bins). This representation allows the planner to encode nonholonomic motion without fully simulating continuous trajectories during the search.

Hybrid A* has found extensive application in ground vehicle and mobile robot navigation since it incorporates simple local motion primitives with discrete search [1], [3], [4].

B. Discrete State Lattice and Motion Primitives

The search space is organized as a 3D lattice consisting of a 2D grid over (x, y) and a discrete heading dimension θ_h . For each expanded node, the planner applies motion primitives that approximate small segments of the robot's kinematic model. Each primitive applies a motion of the form

$$\begin{aligned} x_{k+1} &= x_k + \Delta s \cos(\theta_h), & y_{k+1} &= y_k + \Delta s \sin(\theta_h), \\ \theta_{h,k+1} &= \theta_h + \Delta\theta_h, \end{aligned}$$

where Δs is the grid spacing and $\Delta\theta_h$ corresponds to the selected primitive. This keeps the search computationally reasonable while enabling the planner to consider viable robot motion.

C. Collision Checking With Obstacle Inflation

Before planning, the occupancy grid is processed through obstacle inflation to account for the physical size of the robot. Any cell within a radius,

$$r_{\text{infl}} = r_{\text{robot}} + r_{\text{margin}}$$

of an obstacle is considered occupied. This guarantees that planned routes do not go through small spaces or brush the edges of obstructions. Mobile robot navigation and local avoidance techniques like the Dynamic Windows Approach (DWA) frequently employ inflation-based collision checking. [11].

D. Search Procedure and Cost Structure

Hybrid A* uses the same basic structure as classical A*: a priority queue ordered by

$$f(s) = g(s) + h(s),$$

where $g(s)$ is the accumulated cost of the primitives taken so far, and $h(s)$ is a heuristic estimating the distance to the goal. We use the Euclidean distance as the heuristic:

$$h(s) = \sqrt{(x - x_g)^2 + (y - y_g)^2}.$$

The primitive cost is proportional to its geometric length,

$$c = \|\Delta s\|,$$

which biases the planner toward shorter, more direct paths. Backtracking through the stored predecessor links yields the end path when the planner finds a state that matches the direction and goal position.

E. Generating a Dense Nominal Trajectory

A list of sparse waypoints at grid resolution is returned by Hybrid A*. But the controller needs a more finely sampled and smoother reference. In order to create a dense nominal trajectory, we interpolate the coordinates and headings:

$$x(t), y(t), \theta(t) \quad \text{for } t = 0, \dots, N.$$

Linear interpolation works well for our controller, though spline-based smoothing could be incorporated if desired for tighter curvature control.

F. Role of Hybrid A* in the Overall System

A global reference route that respects obstacle geometry and fundamental nonholonomic restrictions is provided by Hybrid A*. However, the route by itself is insufficient for reliable navigation, as it ignores disruptions or incompatibilities in the robot model. The MPC-iLQR controller, which modifies the robot's mobility in real time to maintain safety and efficiency, is based on the dense nominal trajectory generated by the planner.

This separation of duties is deliberate: MPC-iLQR oversees real-time corrections, smoothness, and replanning triggers when deviations get too great, whereas Hybrid A* maintains long-range planning.

IV. LOCAL TRAJECTORY TRACKING WITH MPC-iLQR

The robot must trace the collision-free path provided by the global planner consistently in spite of noise and discrepancies in the real dynamics. We employ an iterative Linear Quadratic Regulation (iLQR)-based receding-horizon controller for this purpose. In contrast to straightforward feedback techniques like PID, iLQR forecasts the robot's evolution over a brief period of time and modifies its control sequence to remain near the nominal trajectory.

A. Receding-Horizon Formulation

At each step, the controller optimizes over a horizon of N steps. Given the current state x_0 and the nominal trajectory $\{x_t^{\text{nom}}\}$, it solves

$$\min_{\{u_t\}} \sum_{t=0}^{N-1} \left[(x_t - x_t^{\text{nom}})^\top Q (x_t - x_t^{\text{nom}}) + u_t^\top R u_t + c_{\text{obs}}(x_t) \right] + (x_N - x_N^{\text{nom}})^\top Q_f (x_N - x_N^{\text{nom}}).$$

The robot's nominal kinematics are

$$x_{t+1} = f(x_t, u_t) = \begin{bmatrix} x_t + v_t \cos \theta_t dt \\ y_t + v_t \sin \theta_t dt \\ \theta_t + \omega_t dt \end{bmatrix}.$$

B. Linearization and Backward Pass

iLQR approximates the nonlinear problem by linearizing the dynamics around the current rollout:

$$x_{t+1} \approx f(\bar{x}_t, \bar{u}_t) + A_t(x_t - \bar{x}_t) + B_t(u_t - \bar{u}_t),$$

with

$$A_t = \begin{bmatrix} 1 & 0 & -v \sin \theta dt \\ 0 & 1 & v \cos \theta dt \\ 0 & 0 & 1 \end{bmatrix}, \quad B_t = \begin{bmatrix} \cos \theta dt & 0 \\ \sin \theta dt & 0 \\ 0 & dt \end{bmatrix}.$$

A backward Riccati pass calculates the feedforward and feedback terms using this linearized model and quadratic cost approximations:

$$u_t^* = \bar{u}_t + k_t + K_t(x_t - \bar{x}_t).$$

C. Forward Pass and Soft Obstacle Costs

A forward rollout with step size α generates a new candidate trajectory:

$$u_t^{\text{new}} = \bar{u}_t + \alpha k_t + K_t(x_t^{\text{new}} - \bar{x}_t).$$

Obstacle avoidance is handled through the soft penalty,

$$c_{\text{obs}}(x) = \exp\left(-\frac{d(x)}{\sigma}\right),$$

which increases sharply near obstacles and naturally biases the trajectory away from unsafe regions [10].

D. Handling Mismatch and Comparison to PID

Since the simulated robot is affected by velocity scaling errors, heading drift, and additive noise, the overall behavior of the simulated robot is usually different as compared to the nominal model. These deviations are corrected by the feedback term, K_t and the trajectory is re-linearized continuously on the basis of the actual motion of the robot.

For comparison, a PID-like controller is implemented:

$$v = K_p \|x^{\text{nom}} - x\|, \quad \omega = K_\theta (\text{atan2}(y^{\text{nom}} - y, x^{\text{nom}} - x) - \theta).$$

While PID often produces smooth motion, it lacks predictive capability and struggles around tight turns or in the presence of significant model mismatch.

E. Summary

In general, MPC-iLQR offers a predictive, disturbance-sensitive controller that follows the nominal trajectory more precisely than PID, especially in cluttered environments or when there is a model mismatch. It is suitable for real-time robotic navigation due to its combination of feedforward optimization and feedback stabilization.

V. SIMULATION RESULTS

To evaluate the full navigation pipeline, we tested the robot on the cluttered map shown earlier using both MPC-iLQR and PID controllers under identical noise, actuator lag, and model mismatch. The Hybrid A* planner produced a nominal global path with 112 waypoints, which was then interpolated and used as the reference trajectory for both controllers.

A. Executed Trajectories Under Noise and Mismatch

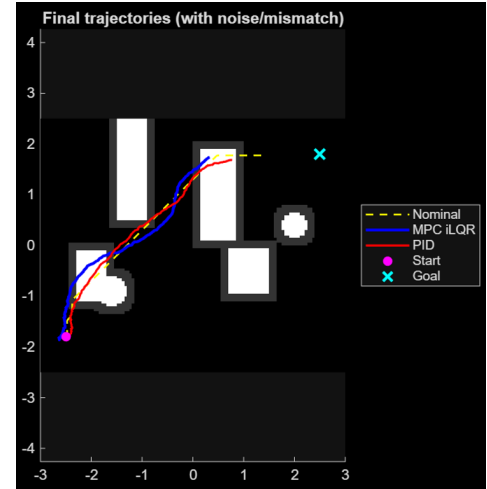


Fig. 1. Executed trajectories of MPC-iLQR and PID under noise and model mismatch, overlaid on the Hybrid A* nominal path.

Fig. 1 compares the nominal Hybrid A* path with the actual trajectories executed by both controllers. MPC-iLQR generally stays closer to the nominal path, especially in narrow corridors, while PID tends to drift outward due to its reactive nature. Both controllers reach the goal, but MPC-iLQR makes

fewer large corrections and maintains tighter adherence to the planned geometry. Soft obstacle costs also cause MPC-iLQR to gently bias its trajectory away from walls, even before the robot is close to them.

B. Velocity and Tracking Behavior Over Time

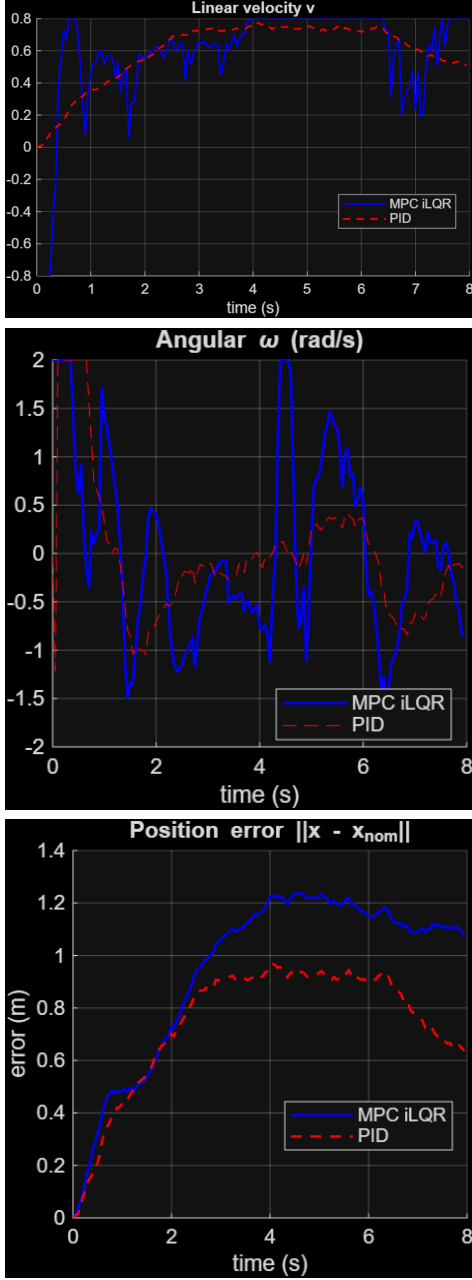


Fig. 2. Time histories of linear velocity, angular velocity, and tracking error for MPC-iLQR and PID.

Fig. 2 shows the time-domain behavior for both controllers. The linear velocity plot indicates that MPC-iLQR makes more aggressive adjustments early on as it attempts to align with the nominal heading, while PID ramps up smoothly and maintains a steadier velocity. The angular velocity plot highlights MPC-iLQR's anticipatory behavior, with sharper corrections

occurring near turns or obstacle boundaries. PID produces smoother control signals but reacts later, which contributes to greater drift from the nominal trajectory. The position-error plot reflects this difference: MPC initially exhibits higher variation due to aggressive corrections, but PID maintains a consistently larger steady-state error and never fully converges.

C. Evolution of MPC Horizon Cost

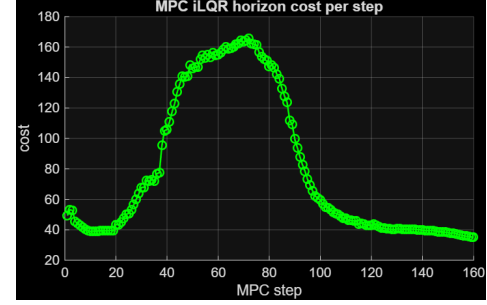


Fig. 3. MPC-iLQR horizon cost at each receding-horizon update step.

The horizon-cost evolution in Fig. 3 provides insight into how the controller responds to the environment. Cost values rise significantly between steps 40 and 80, coinciding with the robot navigating the most congested region of the map. The cost decreases smoothly afterward, reflecting lower tracking difficulty in more open areas. These trends indicate that the MPC controller is sensitive to environmental complexity and that it stabilizes effectively despite noise and mismatch.

D. Quantitative Comparison

The console output summarizing numerical performance is:

```
RMSE position (MPC iLQR): 0.997
RMSE position (PID      ): 0.779
Control effort (MPC): 210.575
Control effort (PID): 142.797
```

PID achieves a lower overall RMSE, but this reflects its tendency to cut corners instead of closely following the nominal path. MPC-iLQR uses more control effort, as expected for a predictive controller that constantly re-optimizes its trajectory. Despite this, MPC-iLQR maintains a safer, more intentional trajectory, particularly near obstacles.

E. Summary

Overall, the results demonstrate that MPC-iLQR provides more robust and anticipatory navigation behavior than PID. The predictive structure of iLQR allows the controller to adjust proactively before entering constrained regions, while PID reacts only to instantaneous error and struggles with accumulated drift. Even under velocity scaling errors, heading drift, actuator lag, and additive noise, MPC-iLQR remains stable and continues to track the nominal path reliably. The combination of Hybrid A* and MPC-iLQR therefore forms a practical and effective framework for real-world differential-drive robot navigation.

VI. DISCUSSION

The combined Hybrid A* and MPC-iLQR framework was an effective option for navigating under noise and model mismatch, according to the simulation findings, and the trials justified a number of the design proposals. Hybrid A* was chosen for its ability to incorporate heading discretization and nonholonomic structure, providing reliable collision-free paths even in narrow regions. MPC-iLQR served as the local controller due to its predictive capability, soft obstacle costs, and ability to reoptimize under mismatch. Performance differences between MPC-iLQR and PID confirm these design choices: MPC-iLQR anticipates turns and rejects disturbances more effectively, while PID remains smoother but less accurate.

A number of unanticipated behaviors surfaced in this project. The enforced velocity-scaling mismatch caused the true robot to advance more quickly than the model, and at first we thought the MPC horizon would account for the majority of this difference. Rather, MPC occasionally showed brief spikes in horizon cost (Fig. 3) when large disturbances moved the system far off the normal rollout, whereas PID accumulated error much more quickly than expected. These spikes indicated a sensitivity we did not completely expect: iLQR's performance dropped abruptly when the warm-start was inadequate or when the linearization no longer captured local dynamics. Nevertheless, following each disruption, the controller bounced back fast, proving the significance of continuous reoptimization.

Examining the control inputs revealed another trade-off in the design. Despite achieving improved obstacle awareness and lower RMSE, MPC-iLQR's quicker actuation changes might not be as appropriate for hardware with low torque or actuator bandwidth. PID offered smoother and occasionally more energy-efficient control, despite its lower accuracy. MPC-iLQR was chosen for robustness and predictive behavior rather than simplicity, which is seen in this difference.

Overall, the choices made in the planning and control architecture were validated by the results. Hybrid A* reliably produced feasible global paths, and MPC-iLQR provided strong robustness, accuracy, and disturbance rejection, while PID revealed the limitations of purely reactive tracking. The unanticipated characteristics, notably cost spikes and susceptibility to significant deviations, point to areas where the framework may be strengthened in the future by adaptive warm-starting or more robust MPC variations.

VII. CONCLUSION

This project implemented a complete navigation pipeline that combined Hybrid A* global planning with MPC-iLQR local control and evaluated the system under noise and model mismatch. Hybrid A* generated feasible, obstacle-aware paths, while MPC-iLQR refined the trajectory through predictive optimization and continuous replanning. A PID controller served as a baseline, allowing us to compare predictive and reactive strategies under identical disturbances. From the simulations, we learned that MPC-iLQR offers stronger robustness and anticipatory behavior, whereas PID provides smoother but

less accurate tracking. Overall, the work demonstrated that integrating Hybrid A* with MPC-iLQR forms an effective and reliable approach for differential-drive navigation in uncertain environments.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [2] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, 1986.
- [3] D. Dolgov *et al.*, "Path planning for autonomous vehicles in unknown semi-structured environments," in *ICRA*, 2008.
- [4] M. Pivtoraiko and A. Kelly, "Efficient constrained path planning via search in state lattices," in *ICRA*, 2005.
- [5] F. Borrelli *et al.*, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.
- [6] E. F. Camacho and C. Bordons, *Model Predictive Control*. Springer, 2013.
- [7] Y. Tassa *et al.*, "Control-limited differential dynamic programming," in *ICRA*, 2014.
- [8] M. Morari and J. H. Lee, "Model predictive control: past, present and future," *Computers & Chemical Engineering*, 1999.
- [9] B. Kouvaritakis and M. Cannon, *Model Predictive Control: Classical, Robust, and Stochastic*. Springer, 2015.
- [10] A. Richards and J. P. How, "Aircraft trajectory planning with collision avoidance using mixed integer linear programming," in *ACC*, 2002.
- [11] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.